



TraceDev: A Traceability-Driven Multi-agent Framework for Requirement-to-Code Development

MINGYU CHEN[†], Harbin Institute of Technology, Shenzhen, China
YAKUN ZHANG[†*], Harbin Institute of Technology, Shenzhen, China
ZIHAO XIE, Harbin Institute of Technology, Shenzhen, China
YIXING LUO, Beijing Institute of Control Engineering, Beijing, China
JINRUI XU, Harbin Institute of Technology, Shenzhen, China
CUIYUN GAO, Harbin Institute of Technology, Shenzhen, China
KAIQI ZHAO, Harbin Institute of Technology, Shenzhen, China
YUNMING YE, Harbin Institute of Technology, Shenzhen, China

In modern software development, the rapid advancement of Large Language Models (LLMs) has made the end-to-end transformation of Natural Language Requirements (NLRs) into executable repository-level code increasingly feasible. However, existing approaches typically rely on simplified instructions (e.g., single-sentence descriptions), failing to reflect complex software development scenarios. Moreover, they lack explicit requirement traceability mechanisms, making it difficult to precisely align and validate generated code against original requirements. To address these limitations, we propose TraceDev, a multi-agent framework for automated software development grounded in use cases that contain multiple functional points and complex semantics. TraceDev employs five role-specific agents, including a Requirement Refiner, Designer, Developer, Tester, and Validator. Notably, the Validator Agent constructs and maintains a heterogeneous traceability graph that links requirements, design models, and code artifacts for interacting with the preceding four agents. The traceability graph maintains consistency across various artifacts and serves as a structured context for efficient memory management, supporting reliable repository-level code generation. We evaluate TraceDev on two widely used datasets (including 125 use cases) compared with two state-of-the-art approaches. On the ETOUR dataset, TraceDev achieves a success rate of 53.63%, outperforming baseline approaches by up to 186.63%. A similar trend is observed on the SMOS dataset, where TraceDev attains a success rate of 56.82%, exceeding baseline approaches by up to 340.80%. These results demonstrate the effectiveness of TraceDev in repository-level code generation from requirements.

CCS Concepts: • **Software and its engineering** → *Software development techniques*; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Large language models, Multi-agent, Code Generation

[*]Corresponding author; [†] Equal contribution.

Authors' Contact Information: Mingyu Chen, 220110123@stu.hit.edu.cn, Harbin Institute of Technology, Shenzhen, Guangdong, China; Yakun Zhang, zhangyk@hit.edu.cn, Harbin Institute of Technology, Shenzhen, Shenzhen Key Laboratory of Internet Information Collaboration and State Key Laboratory of Novel Software Technology, Nanjing University, Nanjing, China; Zihao Xie, 25s151153@stu.hit.edu.cn, Harbin Institute of Technology, Shenzhen, Shenzhen Key Laboratory of Internet Information Collaboration, Guangdong, China; Yixing Luo, luoyi_xing@126.com, Beijing Institute of Control Engineering, Beijing, China; Jinrui Xu, 220111026@stu.hit.edu.cn, Harbin Institute of Technology, Shenzhen, Guangdong, China; Cuiyun Gao, gaocuiyun@hit.edu.cn, Harbin Institute of Technology, Shenzhen, Guangdong, China; Kaiqi Zhao, zhaokaiqi@hit.edu.cn, Harbin Institute of Technology, Shenzhen, Shenzhen Key Laboratory of Internet Information Collaboration, Guangdong, China; Yunming Ye, yeyunming@hit.edu.cn, Harbin Institute of Technology, Shenzhen, Shenzhen Key Laboratory of Internet Information Collaboration, Guangdong, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA080
https://doi.org/10.1145/3832171

ACM Reference Format:
Mingyu Chen, Yakun Zhang, Zihao Xie, Yixing Luo, Jinrui Xu, Cuiyun Gao, Kaiqi Zhao, and Yunming Ye. 2026. TraceDev: A Traceability-Driven Multi-agent Framework for Requirement-to-Code Development. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA080 (October 2026), 24 pages. https://doi.org/10.1145/3832171

1 Introduction

With the increasing complexity of modern software systems, traditional software development faces significant challenges [13, 43, 53, 55]. The long chain from requirements analysis and system design to coding and testing is not only time-consuming but also error-prone [60]. In recent years, the rapid development of Large Language Models (LLMs), represented by Gemini [2] and DeepSeek [27], has brought a revolutionary paradigm shift to the field of software engineering [12, 14, 24, 32, 59]. These techniques make the vision of "Requirements as Code" increasingly feasible [60]. Consequently, the focus of both industry and academia is shifting from simple code completion to higher-level tasks (i.e., the end-to-end transformation of Natural Language Requirements (NLRs) into *repository-level code generation*) [10, 30, 38, 47, 58].

Existing studies mainly focus on building automated frameworks based on multi-agent collaboration [30, 47], in which different agents play different roles and collaborate to simulate the entire Software Development Life Cycle (SDLC). While these studies have demonstrated progress in the end-to-end transformation of Natural Language Requirements (NLRs) into repository-level code, they still face the following two limitations.

1) Simplified Development Scenarios. In real-world software development, requirements are typically documented in a structured and fine-grained form, among which use cases are one of the most prevalent formats for requirements specification. An empirical study surveying requirement specifications across 12 companies finds that 33% of interviewees adopt use cases as requirement specifications, demonstrating the widespread application of use cases in real-world software projects [21]. Such requirements described in use cases typically contain *multiple functional points and rich semantic constraints*. However, existing studies mainly focus on development scenarios with simple semantics and structures, such as code generation based on single-sentence natural language instructions or incomplete code snippets (e.g., HumanEval [10] and MBPP [5]). *These research scenarios differ substantially from real-world requirements in terms of task complexity.*

2) Absence of Traceability. During the development and maintenance of software systems, there are numerous software artifacts (e.g., requirements, design models, and source code) that need to be managed [23]. Traceability allows requirements to be traced to related design models and code, facilitating a clear understanding of artifact relationships and improving software maintainability [60]. Existing studies rarely consider the critical role of traceability throughout the Software Development Life Cycle. *Without traceability, semantic deviations and functional omissions may arise during cross-stage transformations of various artifacts*, such as requirements, design models, and code implementations. This undermines the completeness, reliability and maintainability of code generation in complex development scenarios.

Our work. To address the preceding two limitations, we propose TraceDev, a novel **Trace**ability-driven multi-agent Framework for requirements-to-code **Dev**elopment. Specifically, TraceDev distinguishes itself from existing studies through two key features: ***1) Suitable for complex development scenarios.*** TraceDev is specifically designed for complex development scenarios, leveraging use cases composed of multiple function points as requirements rather than simple descriptions. It achieves repository-level code generation[1] by comprehending the full scope of the use case, rather than generating isolated functions. ***2) Traceability Graph construction***

[1]Repository-level code generation in this paper refers to generating a system-level implementation for a complete use-case, consisting of multiple collaborating code files.

and utilization. TraceDev constructs a heterogeneous traceability graph linking requirements, design models, and code artifacts during the development process. The traceability graph enables automated detection of unimplemented or incorrectly implemented requirements, *supporting reliable repository-level code generation in complex development scenarios*. Meanwhile, it also serves as a structured context for managing software artifacts, *providing an efficient memory management mechanism for Multi-Agent Systems*.

To support the preceding two features, TraceDev adopts a multi-agent framework composed of role-specific agents. It employs five agents: Requirement Refiner, Designer, Developer, Tester, and Validator, which are responsible for requirement refinement, design modeling, code implementation, testing, and requirement traceability, respectively. These agents work together to simulate the Software Development Life Cycle (SDLC). In particular, the Validator Agent constructs and maintains a traceability graph to preserve consistency across various artifacts. It also provides an efficient shared memory for all agents to support continuous validation and iterative refinement.

To comprehensively assess the effectiveness of TraceDev, we evaluate TraceDev using two widely adopted datasets in the Requirement Engineering (RE) domain, i.e., ETOUR and SMOS [9], which include 125 use cases. We compare TraceDev with two state-of-the-art approaches, ChatDev [47] and MetaGPT [30]. The evaluation covers three complementary dimensions: automatic evaluation, human evaluation, and code-level statistical analysis. For automatic evaluation, two metrics are used: *semantic-coverage rate*, which measures the coverage of semantic implementation, and *success rate*, which measures the functional correctness of implementation. The experimental results show that TraceDev is effective in the end-to-end transformation of Natural Language Requirements (NLRs) into repository-level code. On the ETOUR dataset, TraceDev attains a semantic-coverage rate of 71.72%, surpassing ChatDev by 51.66% and MetaGPT by 75.14%, and a success rate of 53.63%, exceeding ChatDev by 129.19% and MetaGPT by 186.64%. A similar trend is observed on the SMOS dataset, where TraceDev achieves a success rate of 56.82%, surpassing the baselines by up to 340.80%.

The main contributions of our work are summarized as follows:

- To the best of our knowledge, this work is the first to introduce the traceability graph into code generation. The traceability graph maintains consistency across various software artifacts and serves as a structured context for efficient memory management. Furthermore, our work focuses on complex development scenarios, where requirements are described in use cases with multiple functional points and rich semantic constraints.
- We propose TraceDev, a traceability-driven multi-agent framework with five specialized roles: Requirement Refiner, Designer, Developer, Tester, and Validator. These agents collaboratively automate the Software Development Life Cycle. Notably, the Validator Agent maintains a traceability graph and interacts with the other agents, supporting consistency validation and efficient memory management throughout the development process.
- We evaluated TraceDev on two representative datasets. Experimental results demonstrate that TraceDev outperforms the state-of-the-art baseline approaches, effectively generating high-quality repository-level code based on requirements.

2 Motivational Example

In this section, we use ETOUR, an electronic tourism system, as a motivating example to illustrate the characteristics of requirements in complex development scenarios and the challenges arising from the absence of requirement traceability in software development.

2.1 Inadequacy of Existing Benchmarks for Complex Development Scenarios

In the ETOUR system, requirements are documented in the form of use cases. Figure 1(a) presents a representative use case from the ETOUR system, namely `ModifyCulturalHeritage`. This use case is described in a structured and fine-grained form, consisting of `a participating actor, entry conditions, a detailed flow of events, exit conditions, and quality requirements`. A single use case contains multiple functional points and rich semantic constraints. Such use cases are prevalent in complex development scenarios because they provide a structured format for precisely representing functions, flows, and constraints [1].

However, existing approaches mainly focus on simplified development scenarios[30, 47]. As illustrated in Figure 1(b), current state-of-the-art approaches are evaluated on benchmarks that typically feature simple and coarse-grained inputs. For instance, ChatDev [47] introduces the SRDD (Software Requirement Description Dataset) benchmark, which consists of task-oriented software requirement descriptions. An example from the SRDD benchmark is `Enemy_Eliminator`, described as: "`An action game where the player must eliminate a wave of incoming enemy forces using their shooting skills`". Similarly, MetaGPT [30] is evaluated on the SoftwareDev benchmark, which mainly contains simplified software development requirements (e.g., `Create a Snake Game`). In addition, public benchmarks such as HumanEval [10] and MBPP [5] mainly consist of function-level programming tasks, with each task covering only a single functionality.

Both simplified requirement descriptions, such as SRDD and SoftwareDev, as well as programming tasks like HumanEval and MBPP, focus only on single functionalities or coarse-grained tasks. However, these simplified tasks are insufficient to reflect the nature of requirements in complex development scenarios. **Consequently, there is an urgent need to develop approaches capable of handling complex development scenarios with real-world requirements (e.g., comprehensive and lengthy use cases).**

2.2 Absence of Requirement Traceability

Figure 1(c) presents code generated by ChatDev [47] using Gemini-2.5-Flash [17], based on the use case example from the ETOUR dataset shown in Figure 1(a). Although the generated code appears to follow the main execution flow, it fails to fully cover all specified requirements. For instance, the entry condition "`The agency has logged in`" is omitted in the implementation. This observation indicates that existing LLM-based agents fail to ensure requirement completeness.

The root cause of this issue lies in the absence of requirement traceability throughout the generation process. As shown in Figure 1(d), when requirement traceability is absent, the generated code cannot be reliably traced back to individual requirements, making it difficult to detect missing or incorrectly implemented requirements. Especially during cross-stage transformations among heterogeneous software artifacts such as requirements, design, and code, semantic deviations or functional omissions are prone to occur, thereby severely undermining the completeness and correctness of the implementation. In contrast, when requirement traceability is maintained, each requirement can be systematically traced and verified against the generated code, enabling the identification of missing or incorrect implementations and supporting targeted refinement.

Therefore, it is essential to establish requirement traceability to ensure completeness, correctness, and reliability in real-world software development.

3 TraceDev

In this section, we propose TraceDev, a traceability-driven multi-agent framework that enables end-to-end repository-level code generation from Natural Language Requirements (NLRs) described in use cases. We first present the overview of TraceDev and then describe its details.

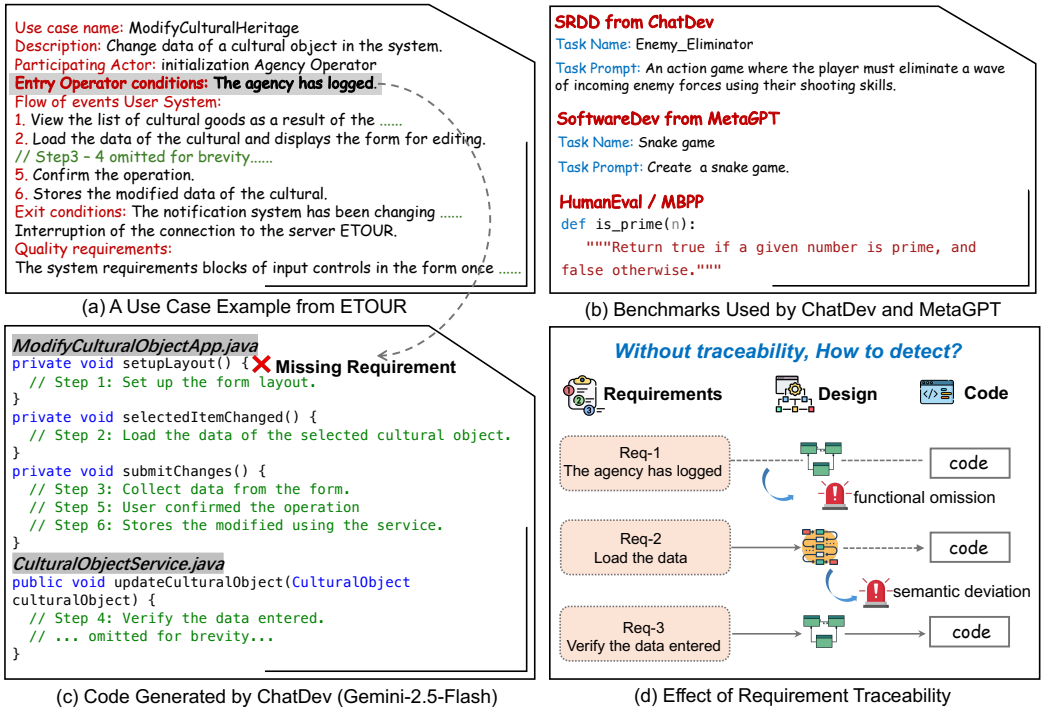


Fig. 1. The motivation example.

3.1 Overview

As illustrated in Figure 2, TraceDev takes a use case as input, which describes the system's behavior under various conditions in response to requests from a user [33]. TraceDev simulates the Software Development Life Cycle by leveraging multiple LLM-based agents. These agents collaboratively transform the refined use case into a series of software artifacts through specialized roles, including a **Requirement Refiner** for eliminating terminology inconsistency and semantic ambiguity in use case, a **Designer** for generating class and sequence diagrams, a **Developer** for code generation, a **Tester** for test case construction and execution, and a **Validator** for maintaining the traceability graph to assess consistency and completeness across different artifacts.

Through the multi-agent collaboration, TraceDev automates the end-to-end software development process, encompassing requirement refinement, system design, code generation, and testing. It iteratively enhances the reliability of artifacts through test execution and the maintenance of a traceability graph, ultimately supporting more automated and reliable software development

3.2 Requirement Refiner Agent

In TraceDev, the Requirement Refiner Agent is responsible for resolving terminological inconsistencies and semantic ambiguities in original use cases. As illustrated in Figure 2, it acts as a preprocessor that takes original use cases as input and produces refined use cases with unified terminology and explicit semantics.

We analyze ETOUR and SMOS, two widely recognized datasets in the requirements domain, and identify ***the following challenges***:

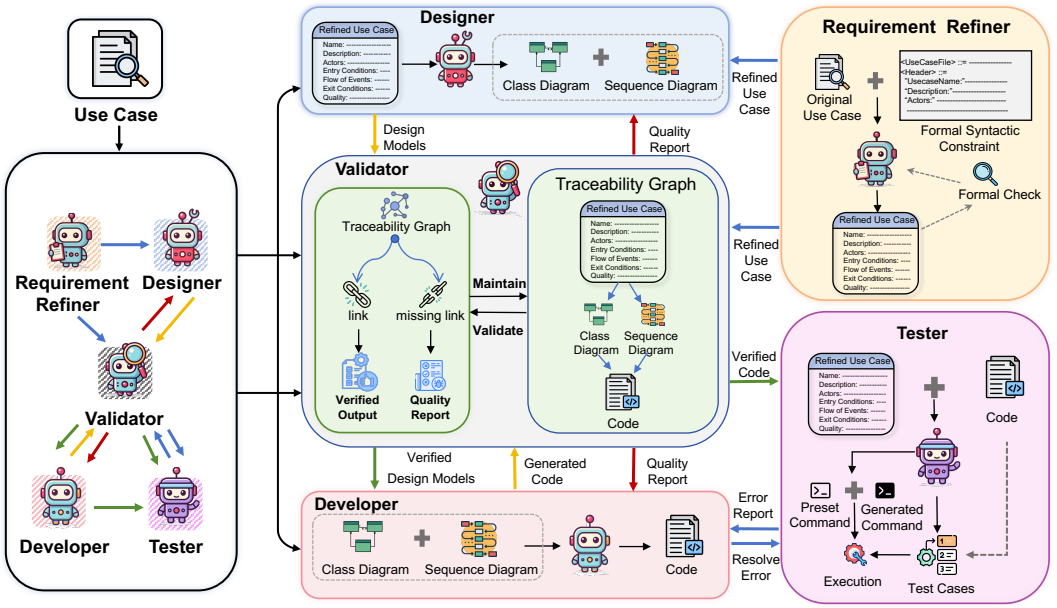


Fig. 2. An overview of TraceDev.

- **Challenge 1: Terminology Inconsistency.** Although the use case specification proposed by Alistair Cockburn has been widely recognized as a standard [16], real-world software systems commonly exhibit variations in the naming of core use case terminology (e.g., names, actors, preconditions, and main flows). As illustrated in Figure 3, when describing the main execution flow, SMOS uses `Events Sequence`, whereas ETOUR uses `Flow of Events`. Similarly, actor fields are labeled as `Actors` by SMOS and `Participating Actor` by ETOUR, respectively. Such terminology inconsistencies limit the applicability of automated tools, particularly when requirements are extracted from a use case as nodes using terminology-based rules or regular expressions.
- **Challenge 2: Semantic Ambiguity.** Even if terminology is aligned, a more severe issue lies in the ambiguity of natural language descriptions. As illustrated in Figure 3, in the `Login` use case from SMOS , Step 1 states, "`Check that username and password have length ≥ 5...`". This sentence omits the acting subject, making it ambiguous whether the validation is performed by the system or by the user. Similarly, in the ETOUR, Step 1 "`View the list of CulturalHeritage...`" lacks an explicit subject (implicitly the actor). Without resolving these ambiguities, the downstream Designer Agent might incorrectly assign responsibilities (e.g., generating system operations within user classes), leading to logical errors in the generated class diagrams and sequence diagrams.

To address the two challenges, the Refiner Agent performs the following two key mechanisms.

(1) Schema Alignment and Subject Inference. As illustrated in Figure 3, a formal syntactic constraint based on Alistair Cockburn's detailed use case specification [16] is injected into the LLM prompt. Guided by this constraint, the LLM performs two tasks in the refinement process. First, it interprets the semantics of the original use case and normalizes the terminology into a standardized schema. For example, terminologies such as "`Entry Operator Conditions`" or "`Preconditions`" are unified into "`Entry Conditions`". Second, the LLM infers missing subjects from contextual

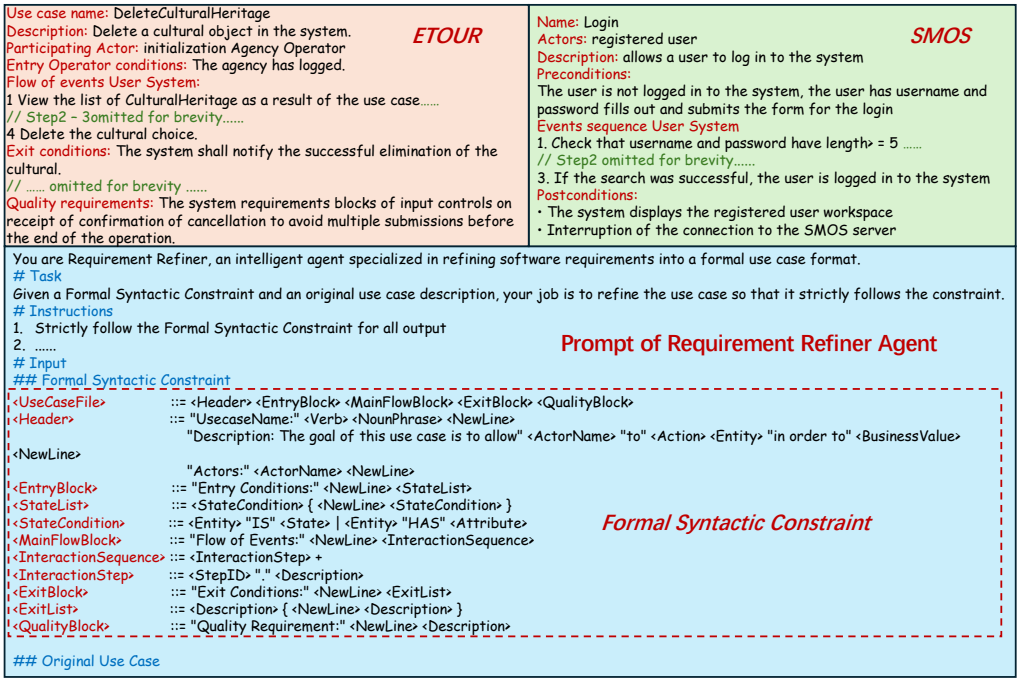


Fig. 3. A comparison of ETour and SMOS use cases, along with the Requirement Refiner Agent's prompt that contains a formal syntactic constraint.

information and rewrites sentences into atomic ⟨Subject + Verb + Object⟩ forms. For example, "Check that..." is transformed into "The system checks that...". By performing schema normalization and subject inference during use case refinement, the Requirement Refiner Agent ensures that the refined use cases are both semantically consistent and grammatically complete.

(2) Iterative Formal Verification. The Refiner Agent employs a syntactic parser to automatically check whether the refined use case satisfies the predefined Formal Syntactic Constraint. If a violation is detected, a feedback loop is initiated to prompt the LLM to regenerate until the output passes the formal check. This mechanism ensures that only structurally valid and unambiguous use cases are passed to downstream agents.

By introducing the Requirement Refiner Agent, we effectively bridge terminological gaps between projects and eliminate semantic ambiguities, ensuring that the downstream Designer Agent can perform modeling based on precise and unambiguous requirements.

3.3 Designer Agent

As illustrated in Figure 2, in TraceDev, the Designer Agent takes the refined use case as input and generates design models, including class diagrams and sequence diagrams.

In real-world software development, software design constitutes a bridge between the requirements and the implementation [56]. Directly converting natural language requirements into programming code often results in code that is poorly structured and difficult to maintain. To address the challenge, we introduce a Designer Agent, which allows TraceDev to establish the architectural skeleton and interaction logic before delving into implementation details. By decoupling structural

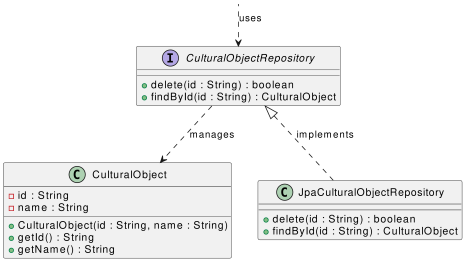


Fig. 4. A class diagram generated by TraceDev using Gemini-2.5-Flash. Due to space limitations, only a subset of the diagram is shown. The diagram is based on the use case from Figure 3.

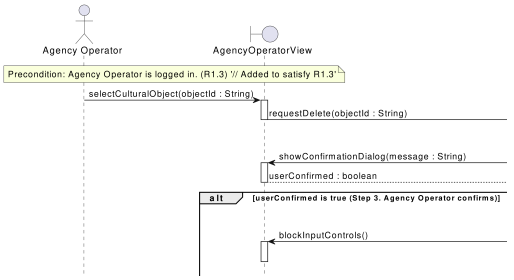


Fig. 5. A sequence diagram generated by TraceDev using Gemini-2.5-Flash. Due to space limitations, only a subset of the diagram is shown. The sequence diagram is based on the use case from Figure 3.

Algorithm 1: Collaborative Code Generation and Verification Process

Input : Refined Use Case $\mathcal{U}$, Design Artifacts $\mathcal{D}$ (Class and Sequence Diagrams), Maximum retry limit K
Output : Verified Executable Code C_{final}

Initialize retry counter $iter \leftarrow 0$;
$C \leftarrow$ GenerateCode($\mathcal{D}$);
$\mathcal{T} \leftarrow$ GenerateTests($\mathcal{U}, C$,);
$iter \leftarrow 0$;
while $iter < K$ **do**
  $pass, \mathcal{L}_{log} \leftarrow$ Execute($C, \mathcal{T}$,);
  if $status = true$ **then**
    // All tests passed **return** C
  $\mathcal{F} \leftarrow$ Extract failure information from $\mathcal{L}_{log}$;
  $C \leftarrow$ SelfCorrect($C, \mathcal{F}, \mathcal{D}$,,)$iter \leftarrow iter + 1$;
return C // Return latest version with warning if max retries reached

definition from code implementation, the Designer Agent can specialize in capturing the structure and interactions of the system, improving the modularity and maintainability of the system.

To provide a complete description of the system's structure and behavior, the Designer Agent generates two complementary UML diagrams in PlantUML code:

- **Class Diagram:** As illustrated in Figure 4, a class diagram defines the static architecture of the system. The LLM analyzes noun phrases within the use case to identify `Classes`, `Attributes`, and `Methods`, as well as `Inheritance` and `Association Relationships`. This ensures that the generated code adheres to object-oriented structural principles.
- **Sequence Diagram:** As illustrated in Figure 5, a sequence diagram depicts the runtime interactions between objects. The LLM analyzes the flow of events in the use case and derives ordered interactions and message flows, thereby ensuring that the designed architectural behavior aligns with the intended execution order of the use case logic.

The Designer Agent bridges the abstraction gap between natural language requirements and code, enabling the LLM to establish a clear architectural skeleton and interaction logic in advance, thereby improving the modularity, structure, and maintainability of the generated code.

3.4 Developer & Tester Agents

As illustrated in Figure 2 and Algorithm 1, after the Designer Agent produces design artifacts, the Developer Agent is responsible for code implementation, while the Tester Agent is responsible for test case generation and execution. Based on the execution results, the two agents collaboratively perform feedback-driven self-correction.

Code Generation Based on Design Artifacts. The Developer Agent takes the class diagrams and sequence diagrams in PlantUML code produced by the Designer Agent as input. These design artifacts are then injected into the prompt, guiding the LLM to generate code whose static structure and dynamic behavior are consistent with the provided UML models (Line 2 in Algorithm 1).

Test Case Generation and Execution. Following code generation, the Tester Agent takes the refined use case and the generated code as input and injects them into a prompt for the LLM. Based on this input, the LLM generates unit test cases that cover both main execution paths and exception handling scenarios. Subsequently, using LLM-generated command lines combined with manually preset commands, the Tester Agent automatically compiles the generated code and executes the test cases (Line 2 & 6 in Algorithm 1).

Self-Correction via Feedback Loop. Following test execution, if any failures are observed, the Tester Agent extracts execution feedback, including error messages and test outcomes (Line 9 in Algorithm 1). This feedback is forwarded to the Developer Agent, which regenerates corrected code (Line 10 in Algorithm 1). This feedback loop iterates with a predefined maximum number of retries to prevent unbounded correction (Line 5 in Algorithm 1).

3.5 Validator Agent

In the software development life cycle simulated by TraceDev, the Refiner, Designer, and Developer Agents sequentially produce diverse artifacts, including refined use cases, design models, and code. However, semantic deviations or functional omissions can easily arise during cross-stage transformations of these heterogeneous artifacts. To address this challenge, the Validator Agent is introduced to construct a traceability graph to maintain the completeness and consistency of the end-to-end development process. As shown in Figure 2, the Validator Agent constructs a heterogeneous **traceability graph** that links the refined use cases from the Requirement Refiner Agent, the design models from the Designer Agent, and the generated code from the Developer Agent.

The traceability graph enables automatic identification of missing links, where requirements lack relevant design elements or design elements lack concrete code implementations. Meanwhile, it also serves as a structured context for managing software artifacts. *Consequently, the traceability graph serves as a robust foundation for large-scale code generation and an efficient memory management mechanism for Multi-Agent Systems.* Based on this graph, the Validator Agent identifies inconsistencies between artifacts, generates a quality report, and passes targeted feedback to the Designer or Developer Agents, thereby guiding iterative refinement.

3.5.1 Traceability Graph. We first introduce how the traceability graph is constructed and then highlight its key benefits in repository-level code generation.

Traceability Graph Construction. To assess traceability completeness, the Validator Agent constructs a graph over heterogeneous software artifacts. The construction process of the traceability graph is detailed in Algorithm 2. Figure 6 presents a traceability graph generated by TraceDev using Gemini-2.5-Flash based on the ETOUR use case from Figure 3. Formally, the traceability graph is defined as a directed graph $G = (V, E)$, where the node set V represents artifact entities, and the edge set E represents traceability links.

Specifically, the node set V is composed of three subsets extracted from different lifecycle stages. For requirements ($\mathcal{U}$), we employ regular expressions to parse the refined use cases to

Algorithm 2: Traceability Graph Construction

input: Refined Use Case $\mathcal{U}$, Design Models $\mathcal{D}$, Generated Code $\mathcal{C}$
output: Traceability Graph $\mathcal{G}(V, E)$

1 $V_{req} \leftarrow$ ExtractNodes($\mathcal{U}$) // Actors, Entry/Exit Conditions, Steps in Flow of Events and Quality Requirement
2 $V_{design} \leftarrow$ ExtractNodes($\mathcal{D}$) // Classes, Methods, Relations, Messages
3 $V_{code} \leftarrow$ ExtractNodes($\mathcal{C}$) // Code Files
4 Initialize $\mathcal{G}$ with $V \leftarrow V_{req} \cup V_{design} \cup V_{code}$ and $E \leftarrow \emptyset$;
5 **foreach** *node* $u \in V_{req}$ **do**
    // Query LLM to identify correlation via constructed prompts
6 $d_{target} \leftarrow$ LLM_SemanticMatch(u, V_{design});
7 **if** $d_{target} \neq \emptyset$ **then**
        // create edge from requirement to design models
8 $E \leftarrow E \cup \{(u \rightarrow d_{target})\}$;
9 **foreach** *node* $d \in V_{design}$ **do**
    // verify syntactic implementation via AST parsing
10 $c_{target} \leftarrow$ AST_SyntacticMatch(d, V_{code});
11 **if** $c_{target} \neq \emptyset$ **then**
        // create edge from design models to generated code
12 $E \leftarrow E \cup \{(d \rightarrow c_{target})\}$;
13 **return** $\mathcal{G}(V, E)$;

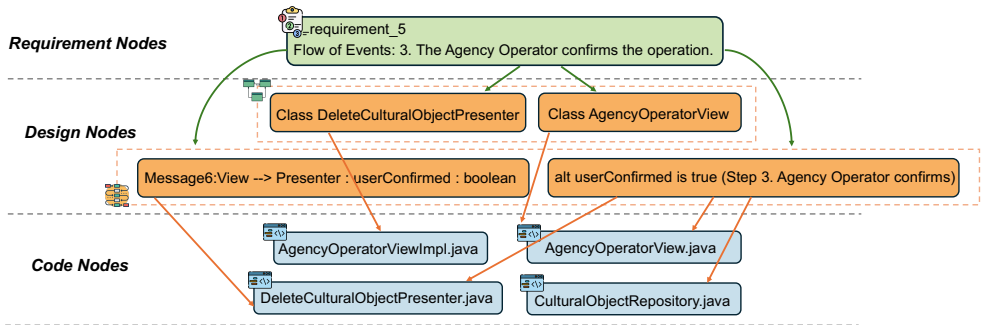


Fig. 6. A traceability graph generated by TraceDev using Gemini-2.5-Flash. Due to space limitations, only a subset of the graph is shown for clarity. The traceability graph is based on the ETOUR use case from Figure 3.

extract semantic entities such as `actors, entry/exit conditions, flow steps, and quality requirements` (Line 1 in Algorithm 2). For design models ($\mathcal{D}$), we analyze the structural representations to identify classes, methods, relations, and messages (Line 2 in Algorithm 2). Finally, for generated code (C), each generated code file is modeled as a node (Line 3 in Algorithm 2). The set V is the union of these extracted subsets, as shown in line 4 of Algorithm 2.

The establishment of traceability between artifacts is divided into two main phases:

(1) Requirement-to-Design Tracing. As shown in lines 5–8 of Algorithm 2, the LLM analyzes each requirement element in the refined use case and identifies relevant design entities from the

Class and Sequence Diagrams. If a relevant logical representation is found in the design models, a directed edge is established from the requirement node to the design node.

(2) Design-to-Code Tracing. As shown in lines 9–12 of Algorithm 2, this phase verifies whether the design elements are implemented in the generated code. Specifically, the Validator Agent uses Abstract Syntax Tree (AST) parsing [54] to analyze the generated code, identifying concrete implementations related to the entities in the design models. When a matching definition or method call is found, a directed edge is established from the design node to the code node.

Memory Management Based on Traceability Graph. Beyond assessing traceability completeness, the traceability graph also serves as a structured context for managing software artifacts. In repository-level code generation, artifacts (e.g., requirements, design models, and code) are often large in scale, making it costly to directly incorporate entire artifacts into the model context. To address this challenge, the traceability graph is used to provide a compressed representation of the context. Specifically, the Validator Agent extracts only the key semantic and structural elements from requirements, design models, and code, and organizes them in the form of nodes and edges. In this way, large-scale artifacts are transformed into a compact and interpretable graph-based representation. This representation preserves the core dependencies among artifacts while significantly reducing the overall context size.

3.5.2 Iterative Refinement via Multi-agent Interaction. To ensure traceability completeness, the Validator Agent interacts with the Designer Agent and the Developer Agent during the design and coding phases, respectively. Specifically, the Validator Agent automatically traverses the traceability graph to identify missing edges, including both requirement-to-design and design-to-code links. These missing links are then injected into prompts for the LLM, which generates a quality report highlighting potential defects. The report is subsequently provided as targeted feedback to the Designer Agent and the Developer Agent to guide corrections in the design and code artifacts. This process is iterated with a maximum number of iterations enforced to prevent infinite loops, enabling progressive improvement of traceability completeness across various artifacts.

4 Experimental Setup

To comprehensively evaluate TraceDev, we answer research questions focusing on three key perspectives. First, we assess the effectiveness of TraceDev by comparing it with two state-of-the-art multi-agent approaches on two representative datasets, using Gemini-2.5-Flash [17] and DeepSeek-V3.2 [18] as foundation models. These two models represent state-of-the-art coding and reasoning performance during the experimental period while offering complementary cost-efficiency. Second, we analyze the contribution of each agent within TraceDev through ablation studies to understand their individual impacts on overall performance. Third, we evaluate the efficiency of TraceDev in comparison with the two baseline approaches. Details are as follows.

RQ1: How effective is TraceDev compared with state-of-the-art multi-agent approaches?
RQ2: How does each agent contribute to the overall performance of TraceDev?
RQ3: How efficient is TraceDev compared to baseline approaches?

4.1 Datasets

Our evaluation is conducted using two real-world system datasets, eTour and SMOS. Table 1 represents a summary of the two datasets. The two selected datasets are provided by the Center of Excellence for Software & Systems Traceability (CoEST) [9] and cover different application domains, including education and tourism. Each dataset contains multiple use cases, and each use case is implemented by multiple code files that together form a code repository.

Following prior works [41, 62], we leverage the use cases and ground-truth implementations provided by the datasets. Based on these, we construct evaluation tasks using test case generation

Table 1. The datasets used for evaluation.

Dataset	Domain	Requirement		Evaluation
		Type	Number	Test Case
eTour	Tourism	Use Case	58	962
SMOS	Education	Use Case	67	1179

```
You are a senior software testing expert.
# Task
Given the **use case** and **Java code**, generate structured **test case suites** in JSON format.
# Instructions:
1.Ensure statement coverage: every statement in the Java class is exercised by at least one test case.
2.Ensure branch coverage: every conditional branch is tested.
3.Ensure functional coverage: every function described in the use case is tested by at least one test case.
3.All fields must be natural language only, no Java code or line references.
4.Output only a valid JSON array, no extra text or explanations.
5.Include a variety of test types: typical cases, edge cases, and invalid/exception-triggering cases.
# Input
Use Case: {usecase_text}
Java Class: {java_code}
# Output Format
[
  { "test_name": "a meaningful and unique name for the test case",
    "purpose": "a concise description of the test intent",
    "input_data": "a natural-language description of all relevant setup data, initial conditions, or input values",
    "procedure": "a natural-language description of the steps executed during the test",
    "expected_result": "the intended functional outcome of the test"',
    "semantic_explanation": "a natural-language explanation of the test case"
  } ,
 ....
]
Begin generate structured **test case suites** in JSON format now.
```

Fig. 7. The prompt for LLM-guided test case generation using use cases and ground-truth code.

to assess the semantic and functional consistency between the generated code and the ground-truth code. Specifically, in the ETOUR and SMOS datasets, we first inject the use cases along with the relevant ground-truth code into the prompt (see Figure 7), guiding the LLM in generating test cases. The generated test cases are designed to achieve statement and branch coverage and to test all functional points in the use case, providing a comprehensive verification of the system's functional behavior. To reduce the risk of test incorrectness, we retain the test cases that pass on ground-truth code. In total, 962 test cases are generated for ETOUR and 1,179 for SMOS.

4.2 Baselines

To demonstrate the effectiveness of TraceDev, we compare it with the following baselines.

- **ChatDev** [47] is a communicative multi-agent framework that leverages a "chat chain" and role specialization to enable LLM agents to cooperatively execute core software lifecycle phases, including design, implementation, and testing.
- **MetaGPT** [30] is a multi-agent framework that assigns agents professional roles (e.g., Product Manager, Architect, Engineer, and QA Engineer) and embeds standardized operating procedures (SOPs) into agent collaboration to guide software development workflows.

4.3 Metrics

To comprehensively evaluate the performance of TraceDev and the selected baselines, we focus on three complementary dimensions, which are automatic evaluation, human evaluation and statistical analysis. Details are as follows.

4.3.1 Automatic Evaluation. Following prior work [41], we adopt two metrics to evaluate generated code from both **semantic presence** and **functional correctness**. The evaluation is conducted on the generated test cases, consisting of 962 tasks in eTour and 1,179 tasks in SMOS. Details of the test case generation process can be found in Section 4.1. The evaluation metrics are as follows:

- **Semantic-coverage Rate** measures how well functions are semantically implemented. This metric verifies whether the generated code logically implements the target functionality. Specifically, we adopt a LLM-as-a-Judge mechanism. To mitigate the variability of single-round judgments, we perform three independent rounds of judgment by the LLM and determine the final decision through majority voting. If the majority of judgments indicate correctness, the generated code is considered to implement the target semantics; otherwise, it is considered incorrect. This approach effectively reduces occasional errors and hallucinations in model evaluation, ensuring the stability of semantic judgments.
- **Success Rate** measures the functional correctness of implementation. For test cases that pass semantic verification, we adapt and execute them against the generated code to evaluate functional correctness. This aligns the evaluation with realistic software testing practices.

Additionally, to ensure fairness and reproducibility of the evaluation, we develop automated execution scripts to drive the entire evaluation process. DeepSeek-V3.2 [18] is used as the foundation model for automated evaluation due to its high cost-effectiveness and computational efficiency.

4.3.2 Human Evaluation. While automated evaluation provides quantitative metrics, it remains insufficient to fully reflect the code's executability and alignment with the requirements. Therefore, following prior works [30, 62], we use human evaluation to further assess the performance of the generated code, focusing on the following metric:

The metric **Executability** evaluates whether the generated code is completely incorrect or meets the requirements to some extent. Each code sample is rated on a scale from 1 to 4:

- 1: The generated code fails to compile or cannot execute.
- 2: The generated code executes but may not meet most workflow requirements.
- 3: The generated code executes and mostly aligns with the requirements.
- 4: The code executes and perfectly satisfies all requirements.

For human evaluation, we invite three experienced software developers with at least four years of programming experience to participate in the assessment. Each participant independently reviews the generated code and evaluates it according to a predefined set of criteria. To reduce individual bias, we provide detailed evaluation guidelines and aggregate the final results by averaging the scores across all evaluators.

4.3.3 Statistical Analysis. We analyze the scale and statistics of the generated code to distinguish minimal solutions from more complete, realistic implementations. The metrics are as follows:

- **File Count:** the number of valid code files, reflecting the modularity of the code.
- **Normalized LOC:** effective lines of code after removing comments, docstrings, and blank lines, used to measure implementation size.
- **Code Token Count:** the number of tokens in the normalized code, measured using a standard tokenizer, reflecting the lexical complexity and richness of implementation details.

Table 2. Evaluation results of baselines and TraceDev on two datasets using Gemini-2.5-Flash and Deepseek-V3.2. "Cov." denotes semantic-coverage rate and "Success" denotes success rate. "Files", "LOC", and "Tokens" denote the number of code files, normalized lines of code, and code token count, averaged per use case.

Dataset	Model	Approach	Automatic Evaluation		Human Evaluation	Statistical Analysis		
			Cov.(%)	Success(%)	Executability	Files	LOC	Tokens
ETOUR	Gemini-2.5-Flash	ChatDev	47.29%	23.40%	2.8	4.81	308.21	2274.47
		Metagpt	40.95%	18.71%	2.26	4.88	253.00	1873.41
		TraceDev	**71.72%**	**53.63%**	**3.60**	12.35	408.77	3278.81
SMOS	Gemini-2.5-Flash	ChatDev	42.49%	24.74%	2.46	5.57	342.63	2543.30
		Metagpt	37.40%	12.89%	2.26	4.79	288.96	2171.52
		TraceDev	**70.05%**	**56.82%**	**3.66**	13.75	427.37	3350.27
ETOUR	DeepSeek-V3.2	ChatDev	41.58%	16.52%	2.9	4.07	380.83	2720.16
		Metagpt	37.01%	20.01%	2.46	3.60	463.64	3503.79
		TraceDev	**54.78%**	**47.19%**	**3.63**	15.11	416.02	2564.47
SMOS	DeepSeek-V3.2	ChatDev	31.46%	16.80%	2.6	3.75	330.75	2377.90
		Metagpt	33.41%	16.01%	2.03	2.53	450.94	3374.47
		TraceDev	**58.60%**	**44.78%**	**3.66**	14.76	403.45	2444.03

4.4 Implementation Details

For the baseline approaches, we directly run their publicly available open-source implementations, making only the necessary adaptations to the LLM API interfaces while keeping all other implementations and parameter settings unchanged to ensure a fair comparison. For temperature configuration, we follow the original settings of each framework whenever available. Specifically, ChatDev uses a temperature of 0.2, TraceDev uses a temperature of 0 to improve reproducibility, and MetaGPT adopts its dynamic temperature mechanism. Following prior work [36], we additionally set the temperature to 0 for DeepSeek-V3.2 during automated evaluation to reduce output randomness.

The maximum number of iterations for repairing missing traceability links is set to three, and up to five remediation attempts are allowed for test failures. These values are determined empirically: we conduct experiments on a subset of the datasets (about 20% of the use cases) and find that these settings achieve a good balance between repair effectiveness and computational cost. All experiments are conducted on a server equipped with two Intel Xeon Silver 4310 CPUs, 1 TB of RAM, running Ubuntu 22.04.5 LTS, and six NVIDIA RTX A6000 GPUs each with 48 GB memory.

5 Evaluation

5.1 Effectiveness of TraceDev (RQ1)

Experimental Design. To evaluate the effectiveness of TraceDev, we compare TraceDev with the baselines (i.e., ChatDev [47] and MetaGPT [30]) on the ETOUR and SMOS datasets using Gemini-2.5-Flash [17] and Deepseek-V3.2 [18]. For human evaluation, we randomly sample 20 use cases (10 cases from each dataset) to conduct a manual assessment of the generated code.

Experimental Results. Table 2 shows that TraceDev outperforms the baselines across various datasets and models. For human evaluation, we adopt Krippendorff's alpha reliability coefficient [29] to measure the inter-rater agreement among multiple evaluators. The overall coefficient is 0.940, indicating excellent agreement. We have three observations from Table 2.

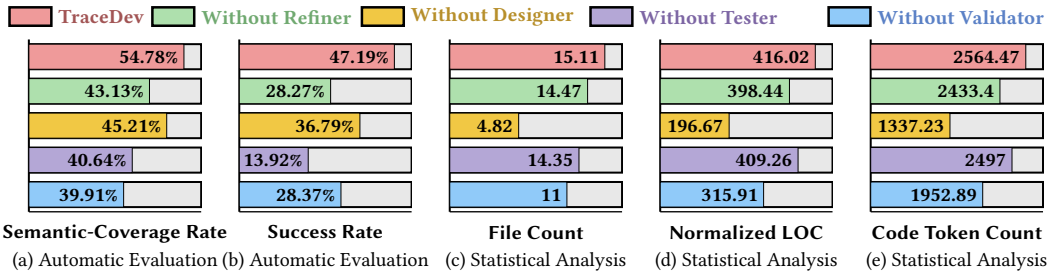


Fig. 8. Ablation study. The "red bar" denotes the original TraceDev, while the "green bar", "yellow bar", "purple bar", and "blue bar" denote TraceDev without the Refiner Agent, without the Designer Agent, without the Tester Agent, and without the Validator Agent, respectively.

(1) In automatic evaluation, TraceDev demonstrates consistent advantages in semantic consistency and functional correctness. Using Gemini-2.5-Flash, it achieves a semantic-coverage rate over 70% and a success rate above 53% on both datasets, significantly outperforming the baselines. This superiority remains when using DeepSeek-V3.2. On the ETOUR dataset, TraceDev attains a success rate of 47.19%, which is more than double the performance of ChatDev (16.52%) and MetaGPT (20.01%). A similar trend is observed on the SMOS dataset, where TraceDev maintains a success rate of 44.78%, whereas the baselines struggle to exceed 17%. These results indicate that TraceDev consistently achieves higher semantic consistency and functional correctness.

(2) In human evaluation, TraceDev demonstrates superior and stable performance in executability. It achieves high executability scores above 3.6 across all datasets and LLMs, where scores above 3 indicate successful execution and strong alignment with requirements. In contrast, the baselines generally score below 3.0. For instance, with Gemini-2.5-Flash on the SMOS dataset, TraceDev attains an executability score of 3.66, representing improvements of 48.8% and 61.9% over ChatDev and MetaGPT, respectively. When using DeepSeek-V3.2, TraceDev consistently maintains an executability score of 3.66, significantly outperforming MetaGPT, which achieves only 2.03.

(3) In statistical analysis, TraceDev generates code that is generally larger in scale and higher in complexity. Specifically, TraceDev produces more code files compared to the baselines. Taking the ETOUR dataset as an example, TraceDev generates over 12 code files per use case, which is more than three times the number produced by the baseline approaches. Regarding code volume, TraceDev generally produces high normalized Lines of code and code token counts. For example, with Gemini-2.5-Flash on the SMOS dataset, TraceDev achieves a high normalized LOC of 427.37 and a lexical complexity of 3,350.27 code tokens, substantially surpassing ChatDev and MetaGPT. These results indicate that TraceDev not only improves functional correctness but also tends to generate more complete and modular implementations, rather than simplified or minimal solutions. Notably, when using DeepSeek-V3.2, MetaGPT produces the largest code volume but only achieves a 16–20% success rate, suggesting that the code quality is low despite its large volume.

Answer to RQ1: TraceDev consistently outperforms baseline approaches in automatic evaluation, human evaluation, and code-level statistical analysis, generating more functionally correct, executable, and comprehensive code.

Table 3. Efficiency evaluation of TraceDev and baseline approaches."Token Usage" denotes the average number of tokens consumed per use case, "Tim" represents the average execution time. "Files", "LoC" and "Code Token" denote the number of code files, normalized lines of code, and code token count, respectively. "Time effect." denotes the time effectiveness. The results for each approach are averaged across all use cases.

Dataset	Model	Approach	Token Usage	Time(s)	Files	Loc	Code Token	Time effect.
ETOUR	Gemini-2.5-Flash	ChatDev	98,766	219.50	4.81	308.21	2274.47	0.11
		Metagpt	219,758	169.30	4.88	253.00	1873.41	0.11
		TraceDev	**109,541**	**385.06**	**12.35**	**408.77**	**3278.81**	**0.14**
SMOS	Gemini-2.5-Flash	ChatDev	109,601	261.94	5.57	342.63	2543.30	0.09
		Metagpt	264,364	174.35	4.79	288.96	2171.52	0.07
		TraceDev	**91,403**	**325.56**	**13.75**	**427.37**	**3350.27**	**0.17**
ETOUR	DeepSeek-V3.2	ChatDev	71,456	587.84	4.07	380.83	2720.16	0.03
		Metagpt	276,610	502.65	3.60	463.64	3503.79	0.04
		TraceDev	**47,234**	**535.26**	**15.11**	**416.02**	**2564.47**	**0.09**
SMOS	DeepSeek-V3.2	ChatDev	62,393	555.53	3.75	330.75	2377.90	0.03
		Metagpt	193,318	327.33	2.53	450.94	3374.47	0.05
		TraceDev	**47,503**	**538.72**	**14.76**	**403.45**	**2444.03**	**0.08**

5.2 Impact of Different Agents in TraceDev (RQ2)

Experimental Design. We perform ablation studies to investigate the necessity of each agent in TraceDev, using the ETOUR dataset and the DeepSeek-V3.2 model.

Experimental Results. Figure 8 presents the ablation results under different agent-removal scenarios. Overall, we find that all agents are essential for achieving the best performance.

(1) The Validator Agent maintains requirement traceability and semantic completeness. Removing the Validator Agent leads to a noticeable drop in semantic-coverage rate, decreasing from 54.78% to 39.91%, the lowest among all variants. This indicates that without the Validator Agent identifying missing links through the traceability graph and facilitating iterative refinement, the generated code is prone to overlooking critical functionality.

(2) The Tester Agent's iterative feedback maintains execution correctness. Removing the Tester Agent results in a dramatic drop in success rate to just 13.9%, the lowest across all variants. Notably, the lines of code (LOC) generated by TraceDev without the Tester Agent are 409.26, very close to the 416.02 lines produced by the full framework. This indicates that simply generating more code is insufficient to guarantee execution correctness. Without the generate-test-refine loop, where the Developer and Tester agents iteratively collaborate to generate, test, and refine code, the code may appear complete but actually contains faults that prevent it from executing correctly.

(3) The Designer Agent maintains structural modularity. In code statistics analysis, TraceDev without the Designer Agent shows the most significant reduction in code scale. The number of code files drops from 15.11 to 4.82, and the normalized line of code (LOC) decreases from 416.02 to 196.67, a reduction of more than half. This demonstrates the crucial role of the Designer Agent in architectural planning. Without its high-level guidance, the system tends to generate simple, non-modular single-file scripts, limiting its ability to handle complex tasks.

Answer to RQ2: All agents are essential for TraceDev. Among all the agents, the Validator Agent maintains semantic completeness, the Tester Agent is critical for execution correctness, and the Designer Agent establishes structural modularity.

5.3 Efficiency of TraceDev (RQ3)

Experimental Design. To assess efficiency, we evaluate execution time and token usage. Following prior works [3, 40], we further calculate *time effectiveness*, defined as the ratio of success rate to execution time, where a high value indicates higher resource utilization efficiency.

Experimental Results. As shown in Table 3, TraceDev maintains consistent efficiency across various datasets and models. Based on Table 3, we achieve the following observations.

(1) In token consumption, TraceDev consistently demonstrates excellent efficiency. with DeepSeek-V3.2 on the ETOUR dataset, TraceDev achieves the lowest token usage across the two datasets, consuming only 47,234 tokens, which reduces token consumption by approximately 33% compared to ChatDev and 83% compared to MetaGPT. Using Gemini-2.5-Flash, TraceDev consumes less than half the tokens of MetaGPT on the SMOS dataset. Notably, with Gemini-2.5-Flash on the ETOUR dataset, TraceDev's total token consumption (109,541) is slightly higher than that of ChatDev (98,766). However, TraceDev demonstrates higher code productivity, requiring only 268 tokens per line of code on average, compared to approximately 320 tokens per line for ChatDev.

(2) In execution time, TraceDev maintains comparable efficiency. Across all datasets and models, TraceDev achieves the highest time effectiveness. Although TraceDev requires more execution time than the baselines when using Gemini-2.5-Flash, this is mainly because it generates a larger number of valuable code tokens. For example, on the SMOS dataset with Gemini-2.5-Flash, TraceDev attains a time effectiveness of 0.17, outperforming ChatDev and MetaGPT by 88.89% and 142.86%. This indicates that the additional time is efficiently transformed into higher-quality code.

(3) The time cost of TraceDev is primarily influenced by the Validator Agent, which constructs a traceability graph across different artifacts and collaborates with the Designer Agent and Developer Agent through multiple iterations of "generate–validate–refine." While introducing computational and interaction costs, this closed-loop mechanism ensures continuous self-correction, allowing TraceDev to enhance the reliability of the entire development process.

Answer to RQ3: TraceDev is superior in token efficiency and maintains comparable execution time to baseline approaches, demonstrating overall strong efficiency.

5.4 Threats to Validity

External Validity. A primary threat to external validity is the generalizability to other datasets and foundation models. To mitigate the threat regarding the dataset, we utilize the ETOUR and SMOS datasets, which are widely recognized in the requirements engineering domain and simulate real-world industrial settings. Regarding model selection, we employed DeepSeek-V3.2 [18] and Gemini-2.5-Flash [17] as the foundation LLMs. These two models are representative in terms of code generation and reasoning capabilities.

Internal Validity. A potential threat to internal validity arises from possible errors in our implementation, prompt design, and experimental execution. To mitigate these threats, we have open-sourced our entire implementation, experimental data, and all prompt templates, facilitating external validation and community feedback for continuous improvement. As for the human evaluation, we invited three experienced developers and provided them with a clear and standardized evaluation process to minimize subjective bias.

Construct Validity. A possible threat to construct validity is related to evaluation metrics. To mitigate this threat, we evaluate the effectiveness of the generated code from three complementary aspects rather than relying on a single metric. Specifically, these aspects include automatic evaluation, human evaluation, and statistical analysis. All the evaluation metrics follow prior

Table 4. Evaluation results of TraceDev and baseline approaches using GPT-5-mini.

Model	Approach	Automatic Evaluation	
		Semantic-coverage Rate(%)	Success Rate(%)
GPT-5-mini	ChatDev	43.06%	7.14%
	MetaGPT	37.96%	25.51%
	TraceDev	**64.69%**	**51.84%**

works [30, 41, 62], providing a reliable and objective basis for assessing the quality of code produced by both TraceDev and the baseline approaches.

6 Discussion

Practical Implications. In requirement-driven software development, traceability plays a crucial role by linking requirements to their corresponding design and implementation [8, 34]. It helps reduce the complexity of various development tasks, such as change impact analysis [20, 28], bug localization [48], and software maintenance [42, 44]. As multi-agent systems gradually gain the ability to generate large-scale software projects, human engineers responsible for subsequent maintenance often face the "cold start" problem [6]. The cold start refers to the difficulty of understanding and operating a new system due to a lack of sufficient context or prior knowledge. Without traceability that links requirements to implementations, understanding the logic of code or locating bugs becomes a high-cognitive-load task [15, 25]. Existing code generation approaches mainly focus on functional correctness but neglect constructing traceability during generation. As a result, although the generated code may be useful, it lacks explicit links to the underlying requirements, which makes later maintenance difficult. However, according to related study[7], coding accounts for only about 20% of the software lifecycle while maintenance occupies roughly 80%. TraceDev provides clear guidance and support for this 80% maintenance phase, substantially reducing the effort required for understanding and iterative development.

Generalizability across LLMs. Our main experiments are conducted on Gemini-2.5-Flash and DeepSeek-V3.2, both of which demonstrate strong capabilities in code generation and reasoning during the evaluation period. To further assess the generalizability of our method, we additionally introduce an OpenAI model, *GPT-5-mini*, for supplementary experiments. Given limited computational resources, we randomly sample 20% subsets from the ETOUR and SMOS datasets for evaluation. As shown in Table 4, TraceDev consistently outperforms all baseline methods using GPT-5-mini, exhibiting stable and consistent performance improvements. Specifically, TraceDev achieves 64.69% on the semantic-coverage rate and 51.84% on the success rate, outperforming the best baseline by 21.63% and 26.33%, respectively. These results indicate that the effectiveness of TraceDev does not depend on any specific large language model backbone, and it generalizes well across different model architectures with strong robustness.

Impact of Traceability Graph Errors. Since TraceDev's iterative refinement relies on the traceability graph constructed by the Validator Agent, its accuracy directly affects the quality of the generated code. Traceability itself is a highly labor-intensive and costly task [49, 50]. Establishing perfectly accurate cross-artifact links in real-world development is inherently difficult, which is why prior work has extensively focused on automated traceability recovery [4, 23, 26, 57]. Accordingly, our goal is not a perfectly accurate graph, but an approximate structure sufficient to support iterative refinement. To this end, TraceDev employs LLM-based semantic matching for requirement-to-design links and AST-based syntactic matching for design-to-code links, with the Validator Agent

automatically detecting missing links and providing feedback for repair. We acknowledge that incorrect links may trigger unnecessary modifications. Through error propagation analysis, we categorize such errors into two types, namely *redundant links* and *incorrect links*. *Redundant links* only trigger additional checks without disrupting overall mappings, whereas *incorrect links* may cause unnecessary modifications. To mitigate such errors, the Validator Agent adopts a bounded repair mechanism, allowing up to three rounds of traceability repair and five rounds of test remediation (Section 4.4), keeping the impact of error propagation within an acceptable range.

Requirement Input Format. TraceDev currently takes use cases as requirement inputs. The structured fields in use cases are used to drive the formal verification of the Requirement Refiner Agent and the extraction of requirement nodes in the traceability graph. Although use cases are widely adopted in industrial practice [21], requirements in real-world software development take diverse forms, including user stories, natural language requirement documents, and software requirements specifications (SRS), which differ in granularity and structure. For such non-use-case requirements, the preprocessing and traceability extraction mechanisms of TraceDev cannot be directly applied, and require redesigning the schema and parsing rules for each specific format.

Lack of Traceability between Code and Tests. The current traceability graph only covers three types of artifacts, namely requirement → design → code, and does not explicitly incorporate the test cases generated by the Tester Agent. As a result, the correspondence between test cases, requirements, and code is not yet explicitly modeled at the graph level. Nevertheless, TraceDev still leverages the feedback loop between the Tester Agent and the Developer Agent to iteratively validate and repair the generated code at the execution level, thereby improving its functional correctness to a certain extent.

7 Related Work

Requirement Granularity in Software Development. In intelligent software development, the reliable transformation of requirements into executable code is an important research topic [10, 31, 35]. Existing studies mainly focus on simple development scenarios, where code is generated from simple, coarse-grained natural language descriptions, such as single-sentence instructions [30, 47] or short function descriptions [5, 10, 31]. However, in real-world software development, requirements often exhibit high complexity and strict logical constraints [31, 45]. To manage this complexity, *Use Cases* have been widely adopted as a standard specification form to structure requirements [16, 22, 33, 51]. Despite their prevalence in development practice, the transformation of structured use cases into executable code remains largely underexplored.

Previous studies usually rely on coarse-grained inputs that do not reflect real-world complex development scenarios. In contrast, TraceDev adopts structured use cases as input to capture complex development scenarios, thereby more closely reflecting real-world software development.

Traceability. Traceability plays a crucial role in software engineering by establishing and maintaining associations among various development artifacts, such as requirements, design models, test cases, and source code [8, 34]. However, maintaining traceability manually is widely considered difficult in development practice due to its high costs in terms of time, budget, and effort [50]. To reduce manual effort, researchers mainly focus on developing automated traceability recovery techniques, among which approaches based on information retrieval (IR) are the most prevalent [4, 26, 57]. These approaches typically identify potential traceability links by measuring similarity between software artifacts such as source code and documentation, with representative techniques including the Vector Space Model (VSM) and probabilistic models [4, 57]. With the advent of large language models (LLMs), recent studies have begun exploring their application in traceability recovery. These studies typically leverage techniques such as model fine-tuning [39], prompt engineering [49], or retrieval-augmented generation (RAG) [23].

Existing studies primarily focus on recovering traceability links after software artifacts (e.g., requirements, design models, and source code) are produced. In contrast, TraceDev automatically constructs a Traceability Graph throughout the development process to support error detection, iterative refinement and long-term maintainability.

Multi-Agent Collaboration in Software Development. With the increasing application of LLMs in software engineering tasks, researchers have gradually recognized the limitations of single-model approaches in complex software development scenarios [19, 61]. Real-world software development typically involves multiple stages, including requirements analysis, system design, code implementation, and test verification. A single LLM struggles to simultaneously satisfy the objectives and constraints of different stages [46, 52]. Consequently, recent studies have introduced multi-agent collaborative frameworks that simulate software development workflows through role specialization [30, 37, 47]. Following this paradigm, TraceDev adopts a multi-agent architecture, in which five specialized agents collaborate to simulate the entire software development lifecycle.

However, existing multi-agent frameworks lack mechanisms to address semantic deviations or functional omissions that can easily arise during cross-stage transformations in the software development lifecycle. To address this challenge, TraceDev constructs a traceability graph, explicitly maintaining traceability between software artifacts (e.g., requirements, design models and code) produced throughout the development process. The graph helps maintain consistency between software artifacts and enhances software reliability.

8 Conclusion

In this paper, we propose TraceDev, a traceability-driven multi-agent framework, which aims to achieve high-quality end-to-end transformation from use cases to executable repository-level code. TraceDev employs five agents, including Requirement Refiner, Designer, Developer, Tester, and Validator, which are respectively responsible for requirement refinement, system design, code generation, testing, and traceability validation. Throughout the development process, the Validator Agent constructs and maintains a heterogeneous Traceability Graph that links requirements, design models, and code artifacts, thereby supporting more reliable software development. We evaluate TraceDev on two real-world requirement datasets (including 125 use cases) and compare it with two state-of-the-art approaches, ChatDev and MetaGPT. Experimental results demonstrate the superior performance of TraceDev. In future work, we plan to extend TraceDev to a wider range of LLMs, additional programming languages, and optimize our prompt template.

Data Availability

We release our source code and data through https://zenodo.org/records/20809458 [11] and maintain the project repository at https://github.com/ISSE-Lab/ISSTA2026-TraceDev. The GitHub repository will host future updates, extended materials, and related follow-up work.

Acknowledgments

We thank all the anonymous reviewers. This work is supported by the National Natural Science Foundation of China (No. 62472126, No. 62272130, No. 62402038, and No. 92582203), the Shenzhen Natural Science Foundation Youth Program (No. QNXMB20250701092047022), and the Shenzhen Science and Technology Program (No. SYSPG20241211173609009).

References

[1] Bente Anda. 2003. *Empirical Studies of Construction and Application of Use Case Models.* Ph. D. Dissertation. ISSN 1501-7710, Nr. 268.

[2] Gemini Google Team: Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2025. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805 [cs.CL] doi:10.48550/arXiv.2312.11805

[3] Anonymous. 2026. ARS Adaptive Reasoning Suppression for Efficient Large Reasoning Language Model. https://openreview.net/forum?id=gTX39XIeMY

[4] Giulio Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. 2025. Recovering Traceability Links Between Code and Documentation: A Retrospective. *IEEE Transactions on Software Engineering* 51, 3 (2025), 825–832. doi:10.1109/TSE.2025.3534027

[5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. doi:10.48550/arXiv.2108.07732

[6] Avijit Bhattacharjee, Banani Roy, and Kevin A. Schneider. 2022. Supporting program comprehension by generating abstract code summary tree. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Pittsburgh, Pennsylvania) *(ICSE-NIER '22)*. Association for Computing Machinery, New York, NY, USA, 81–85. doi:10.1145/3510455.3512793

[7] Barry W. Boehm. 1984. Software Engineering Economics. *IEEE Transactions on Software Engineering* SE-10, 1 (1984), 4–21. doi:10.1109/TSE.1984.5010193

[8] Markus Borg, Per Runeson, and Anders Ardö. 2014. Recovering from a decade: a systematic mapping of information retrieval approaches to software traceability. *Empirical Softw. Engg.* 19, 6 (Dec. 2014), 1565–1616. doi:10.1007/s10664-013-9255-y

[9] Center of Excellence for Software Systems Traceability. 2026. Datasets. http://sarec.nd.edu/coest/datasets.html.

[10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] doi:10.48550/arXiv.2107.03374

[11] Mingyu Chen, Yakun Zhang, Zihao Xie, Yixing Luo, Jinrui Xu, Cuiyun Gao, Kaiqi Zhao, and Yunming Ye. 2026. TraceDev: A Traceability-Driven Multi-agent Framework for Requirement-to-Code Development. doi:10.5281/zenodo.20809458

[12] Xiangping Chen, Xing Hu, Yuan Huang, He Jiang, Weixing Ji, Yanjie Jiang, Yanyan Jiang, Bo Liu, Hui Liu, Xiaochen Li, et al. 2025. Deep learning-based software engineering: progress, challenges, and opportunities. *Science China Information Sciences* 68, 1 (2025), 111102. doi:10.1007/s11432-023-4127-5

[13] Yujia Chen, Mingyu Chen, Cuiyun Gao, Zhihan Jiang, Zhongqi Li, and Yuchi Ma. 2025. Towards Mitigating API Hallucination in Code Generated by LLMs with Hierarchical Dependency Aware. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim, Trondheim, Norway) *(FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 468–479. doi:10.1145/3696630.3728569

[14] Yujia Chen, Yang Ye, Zhongqi Li, Yuchi Ma, and Cuiyun Gao. 2025. Smaller but Better: Self-Paced Knowledge Distillation for Lightweight yet Effective LCMs. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE135 (June 2025), 24 pages. doi:10.1145/3729405

[15] Jane Cleland-Huang, Orlena C. Z. Gotel, Jane Huffman Hayes, Patrick Mäder, and Andrea Zisman. 2014. Software traceability: trends and future directions. In *Future of Software Engineering Proceedings* (Hyderabad, India) *(FOSE 2014)*. Association for Computing Machinery, New York, NY, USA, 55–69. doi:10.1145/2593882.2593891

[16] Alistair Cockburn. 2000. *Writing Effective Use Cases* (1st ed.). Addison-Wesley Longman Publishing Co., Inc., USA. https://dl.acm.org/doi/10.5555/517669

[17] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, Luke Marris, Sam Petulla, Colin Gaffney, Asaf Aharoni, Nathan Lintz, Tiago Pais, Henrik Jacobsson, Idan Szpektor, Nan-Jiang Jiang, Chris Hahn, et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. doi:10.48550/arXiv.2507.06261

[18] DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Zihua Qu, et al. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. doi:10.48550/arXiv.2512.02556

[19] Gustavo de Aquino e Aquino, Nádila da Silva de Azevedo, Leandro Youiti Silva Okimoto, Leonardo Yuto Suzuki Camelo, Hendrio Luis de Souza Bragança, Rubens Fernandes, Andre Printes, Fábio Cardoso, Raimundo Gomes, and Israel Gondres Torné. 2025. From RAG to Multi-Agent Systems: A Survey of Modern Approaches in LLM Development. (2025). doi:10.20944/preprints202502.0406.v1

[20] Davide Falessi, Justin Roll, Jin L.C. Guo, and Jane Cleland-Huang. 2020. Leveraging Historical Associations between Requirements and Source Code to Identify Impacted Classes . *IEEE Transactions on Software Engineering* 46, 04 (April 2020), 420–441. doi:10.1109/TSE.2018.2861735

[21] Xavier Franch, Cristina Palomares, Carme Quer, Panagiota Chatzipetrou, and Tony Gorschek. 2023. The state-of-practice in requirements specification: an extended interview study at 12 companies. *Requirements Engineering* 28, 3 (Sept. 2023), 377–409. doi:10.1007/s00766-023-00399-7

[22] Julian Frattini and Anja Frattini. 2025. Adopting Use Case Descriptions for Requirements Specification: an Industrial Case Study. *arXiv preprint arXiv:2506.13303* (2025). doi:10.1109/RE63999.2025.00026

[23] Dominik Fuchß, Tobias Hey, Jan Keim, Haoyu Liu, Niklas Ewald, Tobias Thirolf, and Anne Koziolek. 2025. LiSSA: Toward Generic Traceability Link Recovery through Retrieval-Augmented Generation. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) *(ICSE '25)*. IEEE Press, 1396–1408. doi:10.1109/ICSE55347.2025.00186

[24] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The Current Challenges of Software Engineering in the Era of Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 127 (May 2025), 30 pages. doi:10.1145/3712005

[25] O.C.Z. Gotel and C.W. Finkelstein. 1994. An analysis of the requirements traceability problem. In *Proceedings of IEEE International Conference on Requirements Engineering*. 94–101. doi:10.1109/ICRE.1994.292398

[26] J. Grundy, J. Hosking, and W.B. Mugridge. 1998. Inconsistency management for multiple-view software development environments. *IEEE Transactions on Software Engineering* 24, 11 (1998), 960–981. doi:10.1109/32.730545

[27] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR* abs/2401.14196 (2024). https://doi.org/10.48550/arXiv.2401.14196

[28] Jin LC Guo, Jan-Philipp Steghöfer, Andreas Vogelsang, and Jane Cleland-Huang. 2025. Natural language processing for requirements traceability. In *Handbook on Natural Language Processing for Requirements Engineering*. Springer, 89–116. doi:10.1007/978-3-031-73143-3_4

[29] Andrew F. Hayes and Klaus Krippendorff. 2007. Answering the Call for a Standard Reliability Measure for Coding Data. *Communication Methods and Measures* 1, 1 (2007), 77–89. doi:10.1080/19312450709336664

[30] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VtmBAGCN7o

[31] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79. doi:10.1145/3695988

[32] Nam Huynh and Beiyu Lin. 2025. Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications. *arXiv preprint arXiv:2503.01245* (2025). doi:10.48550/arXiv.2503.01245

[33] Ivar Jacobson. 1993. *Object-oriented software engineering: a use case driven approach.* Pearson Education India.

[34] Dongming Jin, Zhi Jin, Yiran Zhang, Zheng Fang, Linyu Li, Yuanpeng He, Xiaohong Chen, and Weisong Sun. 2025. UserTrace: User-Level Requirements Generation and Traceability Recovery from Software Project Repositories. doi:10.48550/arXiv.2509.11238

[35] Javed Ali Khan, Shamaila Qayyum, and Hafsa Shareef Dar. 2025. Large Language Model for Requirements Engineering: A Systematic Literature Review. (2025). doi:10.21203/rs.3.rs-5589929/v1

[36] Yukyung Lee, JoongHoon Kim, Jaehee Kim, Hyowon Cho, Jaewook Kang, Pilsung Kang, and Najoung Kim. 2025. CheckEval: A reliable LLM-as-a-Judge framework for evaluating text generation using checklists. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 15771–15798. doi:10.18653/v1/2025.emnlp-main.796

[37] Jierui Li, Hung Le, Yingbo Zhou, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2025. CodeTree: Agent-guided Tree Search for Code Generation with Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3711–3726. doi:10.18653/v1/2025.naacl-long.189

[38] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. arXiv:https://www.science.org/doi/pdf/10.1126/science.abq1158 doi:10.1126/science.abq1158

[39] Jinfeng Lin, Yalin Liu, Qingkai Zeng, Meng Jiang, and Jane Cleland-Huang. 2021. Traceability Transformed: Generating more Accurate Links with Pre-Trained BERT Models. In *Proceedings of the 43rd International Conference on Software Engineering* (Madrid, Spain) *(ICSE '21)*. IEEE Press, 324–335. doi:10.1109/ICSE43902.2021.00040

[40] Junhong Lin, Xinyue Zeng, Jie Zhu, Song Wang, Julian Shun, Jun Wu, and Dawei Zhou. 2026. Plan and Budget: Effective and Efficient Test-Time Scaling on Reasoning Large Language Models. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=ctspw4CqbS

[41] Jane Luo, Xin Zhang, Steven Liu, Jie Wu, Jianfeng Liu, Yiming Huang, Yangyu Huang, Chengyu Yin, Ying Xin, Yuefeng Zhan, Hao Sun, Qi Chen, Scarlett Li, and Mao Yang. 2026. RPG: A Repository Planning Graph for Unified and Scalable Codebase Generation. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=VAQq3Y8tIF

[42] Patrick Mäder and Alexander Egyed. 2015. Do developers benefit from requirements traceability when evolving and maintaining a software system? *Empirical Softw. Engg.* 20, 2 (April 2015), 413–441. doi:10.1007/s10664-014-9314-z

[43] Pedro Neves Mata, José Moleiro Martins, and João Carlos Ferreira. 2025. New software product development: Bibliometric analysis. *Journal of the Knowledge Economy* 16, 1 (2025), 4161–4184. doi:10.1007/s13132-024-02095-5

[44] Patrick Mäder and Alexander Egyed. 2012. Assessing the effect of requirements traceability for software maintenance. In *2012 28th IEEE International Conference on Software Maintenance (ICSM)*. 171–180. doi:10.1109/ICSM.2012.6405269

[45] JJ Norheim, E Rebentisch, D Xiao, L Draeger, A Kerbrat, and OL De Weck. 2024. Challenges in applying large language models to requirements engineering tasks. Des. Sci. 10 (2024). doi:10.1017/dsj.2024.8

[46] Roger Pressman. 2009. *Software Engineering: A Practitioner's Approach* (7 ed.). McGraw-Hill, Inc., USA. https://dl.acm.org/doi/10.5555/1593949

[47] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810

[48] Michael Rath, David Lo, and Patrick Mäder. 2018. Analyzing requirements and traceability information to improve bug localization. In *Proceedings of the 15th International Conference on Mining Software Repositories* (Gothenburg, Sweden) *(MSR '18)*. Association for Computing Machinery, New York, NY, USA, 442–453. doi:10.1145/3196398.3196415

[49] Alberto D. Rodriguez, Katherine R. Dearstyne, and Jane Cleland-Huang. 2023. Prompts Matter: Insights and Strategies for Prompt Engineering in Automated Software Traceability. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. 455–464. doi:10.1109/REW57809.2023.00087

[50] Marcela Ruiz, Jin Yang Hu, and Fabiano Dalpiaz. 2023. Why don't we trace? A study on the barriers to software traceability in practice. *Requirements Engineering* 28, 4 (2023), 619–637. doi:10.1007/s00766-023-00408-9

[51] Kiran Prakash Sawant, Suman Roy, Srivibha Sripathi, François Plesse, and A. S. M. Sajeev. 2014. Deriving requirements model from textual use cases. In *Companion Proceedings of the 36th International Conference on Software Engineering* (Hyderabad, India) *(ICSE Companion 2014)*. Association for Computing Machinery, New York, NY, USA, 235–244. doi:10.1145/2591062.2591193

[52] Ian Sommerville. 2016. *Software Engineering* (tenth ed.). Pearson. https://www.pearson.com/us/higher-education/program/PGM35255.html

[53] Thomas Stober and Uwe Hansmann. 2010. *Traditional Software Development*. Springer Berlin Heidelberg, Berlin, Heidelberg, 15–33. doi:10.1007/978-3-540-70832-2_2

[54] Tree-sitter. 2026. https://github.com/tree-sitter/tree-sitter

[55] Félix Témolé and Desislava Atanasova. 2025. An Integrated Approach to Managing Software Quality in Complex Systems. *American Journal of Software Engineering and Applications* 13, 1 (2025), 1–17. arXiv:https://article.sciencepublishinggroup.com/pdf/10.11648.j.ajsea.20251301.11 doi:10.11648/j.ajsea.20251301.11

[56] Jonathan Ullrich, Matthias Koch, and Andreas Vogelsang. 2025. From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering. *2025 IEEE 33rd International Requirements Engineering Conference (RE)* (2025), 257–266. https://api.semanticscholar.org/CorpusID:280294299

[57] Hongyan Wan, Xinyu He, Yang Deng, and Bangchao Wang. 2025. A systematic mapping study of information retrieval-based requirements traceability methods. *Inf. Process. Manage.* 62, 6 (Nov. 2025), 51 pages. doi:10.1016/j.ipm.2025.104287

[58] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.

[59] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101. doi:10.1007/s11432-024-4222-0

[60] Mohammad Zadenoori, Jacek Dąbrowski, Waad Alhoshan, Liping Zhao, and Alessio Ferrari. 2025. Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. doi:10.48550/arXiv.2509.11446

[61] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2023. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223* (2023). doi:10.1007/s11432-025-4670-0

[62] Sai Zhang, Zhenchang Xing, Ronghui Guo, Fangzhou Xu, Lei Chen, Zhaoyuan Zhang, Xiaowang Zhang, Zhiyong Feng, and Zhiqiang Zhuang. 2025. Empowering Agile-Based Generative Software Development through Human-AI Teamwork. *ACM Trans. Softw. Eng. Methodol.* 34, 6, Article 156 (July 2025), 46 pages. doi:10.1145/3702987

Received 2026-01-30; accepted 2026-04-16